

期末复习 PPT 名词解释

目录

使用说明	2
一、词法分析	2
二、自顶向下分析	3
三、自底向上分析	4
四、语义分析	5
五、中间代码与 IR	7

使用说明

本文件用于背诵边角概念和简答题。每个名词按卷面可写的短定义整理。

一、词法分析

词素

源程序中实际出现、能够匹配某个词法单元模式的字符序列。例如变量名 `count` 是一个词素。

词法单元

词法分析器输出的记号类别，例如 `id`、`num`、`if`。一个词法单元可以对应多个不同词素。

正则表达式

用于描述词法单元模式的形式化表达式。编译器词法分析阶段常用正则表达式描述 `token` 的结构。

NFA

非确定有限自动机。一个状态在同一输入符号上可以有多个后继，也可以有 `epsilon` 转移。

DFA

确定有限自动机。任一状态在任一输入符号上最多只有一个后继状态，执行时只需维护一个当前状态。

epsilon-closure

从某个状态或状态集合出发，只经过 `epsilon` 边能够到达的所有状态集合。

子集构造法

把 NFA 的状态集合看成 DFA 的一个状态，从而把 NFA 转换成等价 DFA 的方法。

DFA 最小化

把等价状态合并，使 DFA 状态数减少，同时保持识别语言不变。

二、自顶向下分析

递归下降分析

一种自顶向下语法分析方法。每个非终结符通常对应一个分析过程。

回溯

当分析器选择某条产生式失败后，退回之前位置并尝试另一条产生式的过程。

左递归

若非终结符 A 能推出以 A 开头的串，则存在左递归。直接左递归形如 $A \rightarrow A\alpha$ 。

左公因子

同一非终结符的多个产生式右部具有相同开头时，这个共同开头称为左公因子。

FIRST 集

$\text{FIRST}(\alpha)$ 是 α 能推出的串的第一个终结符集合；若 α 能推出空串，则包含 ϵ 。

FOLLOW 集

$\text{FOLLOW}(A)$ 是在某些句型中可能紧跟在非终结符 A 后面的终结符集合。

LL(1) 文法

从左到右扫描输入，构造最左推导，并只向前看一个输入符号即可选择产生式的文法。

预测分析表

LL(1) 分析使用的表。行是非终结符，列是终结符或 $\$$ ，表项填入应使用的产生式。

三、自底向上分析

自底向上分析

从输入串开始，不断把产生式右部规约为左部，最终规约到开始符号的分析方法。

句柄

在自底向上分析中，当前句型中下一步应该被规约的产生式右部。

最右推导的逆过程

自底向上分析的归约序列等价于某个最右推导反过来执行。

LR(0) 项

在产生式右部加入一个点的项目，用于表示当前已经识别到产生式的哪个位置。

规范 LR(0) 项集族

由 closure 和 GOTO 构造出的全部 LR(0) 状态集合。

closure

若项集中某个项目的点后面是非终结符，则把该非终结符的产生式以点在最前的形式加入项集，直到不能继续加入。

GOTO

$GOTO(I, X)$ 表示在项集 I 中读入文法符号 X 后，点越过 X 并做 closure 得到的新项集。

ACTION 表

LR 分析表中处理终结符的部分，用于决定移进、规约、接受或报错。

GOTO 表

LR 分析表中处理非终结符的部分，用于规约出非终结符后决定转入哪个状态。

移进-规约冲突

同一状态、同一输入符号下，分析器既可以移进，又可以规约，无法唯一决定动作。

规约-规约冲突

同一状态、同一输入符号下，分析器可以按两条或多条产生式规约，无法唯一决定动作。

内核项

LR(0) 项集中，初始项 $S' \rightarrow \cdot S$ 以及所有点不在产生式最左端的项。

非内核项

除初始项 $S' \rightarrow \cdot S$ 外，点在产生式最左端的项。

SLR

Simple LR。它在 LR(0) 项集族基础上使用 FOLLOW 集限制规约动作。

SLR 的弱点

FOLLOW 集只与非终结符有关，不区分具体 LR 状态上下文；FOLLOW 集过大时仍可能产生冲突。

四、语义分析

SDD

语法制导定义。它给文法符号关联属性，并用语义规则描述属性如何计算。

SDT

语法制导翻译。它把语义动作嵌入产生式，描述分析过程中何时执行动作。

属性

与文法符号关联的语义信息，例如类型、值、代码、符号表条目等。

综合属性

由子结点或当前结点信息计算，并向父结点传递的属性。

继承属性

由父结点或左侧兄弟结点传给当前结点的属性。

S 属性定义

只包含综合属性的 SDD。属性信息只从子结点向父结点传递。

L 属性定义

继承属性只能依赖父结点或左侧兄弟结点信息的一类 SDD，适合自左向右求值。

依赖图

表示属性实例之间依赖关系的图。若属性 a 的计算依赖属性 b ，则图中有从 b 到 a 的边。

拓扑序

在无环依赖图中，满足所有依赖先于被依赖对象计算的一种顺序。

属性文法

没有副作用、只通过属性和语义规则描述语义计算的文法扩展。

语义动作

嵌入产生式中的实现动作，例如插入符号表、生成中间代码或输出结果。

副作用

语义动作对属性计算之外的外部状态进行修改，例如修改符号表或输出代码。

受控副作用

执行位置和顺序明确，并且不会破坏属性依赖关系的副作用。

CST

具体语法树，完整反映文法推导过程，包含全部语法细节。

AST

抽象语法树，去掉冗余语法细节，只保留表达程序语义所需的核​​心结构。

AST 结点表示

AST 结点通常用对象或记录表示。内部结点保存 op 和子结点指针，叶子结点保存词法值或标识符名字。

构造 AST 的 SDD

用语义属性保存子树结点，并在产生式归约或语义动作执行时创建新的 AST 结点。

符号表

记录程序中标识符信息的数据结构，通常包含名字、类型、作用域、存储位置等信息。

作用域

名字绑定有效的程序区域。内层作用域可以遮蔽外层同名标识符。

符号表栈

用于处理嵌套作用域的符号表组织方式。进入作用域时压栈，退出作用域时弹栈。

类型检查

根据符号表和表达式类型规则检查赋值、运算、函数调用等是否类型匹配。

五、中间代码与 IR

IR

中间表示。编译器前端和后端之间使用的程序表示形式。

高层 IR

保留较多源语言结构的 IR，例如循环、函数、对象等结构仍然显式存在。

中层 IR

介于源语言结构和机器细节之间的 IR。典型形式包括三地址码、四元式、SSA 形式的三地址码。

低层 IR

接近目标机器的 IR，例如显式跳转、加载/存储、寄存器或栈操作。

树形 IR

以树结构表示程序，例如 AST。适合前端和早期高层变换，不适合直接做低层优化。

堆栈式 IR

操作数隐含在栈上的 IR，例如 JVM 字节码。形式紧凑，但数据依赖不如三地址码显式。

强类型 IR

在 IR 中保留较精确的类型、对齐或内存访问信息，便于检查和优化。

弱类型 IR

IR 中类型信息较少或较粗略，表示更接近低层机器操作。

CFG

控制流图。结点通常是基本块，边表示程序执行时可能发生的控制转移。

DFG

数据流图。用于表示值的定义、使用和数据依赖关系。

三地址码

一种常见中间代码形式，每条指令右侧最多包含一个运算符，常见形式为 $x = y \text{ op } z$ 。

四元式

三地址码的一种表格表示，字段为 (op, arg1, arg2, result)。

三元式

三地址码的一种表示，字段为 (op, arg1, arg2)，结果通过该条指令的位置编号引用。

间接三元式

在三元式外增加指针表，通过调整指针表控制指令顺序，避免移动三元式本体。

param

函数调用三地址码中用于传递实参的指令。

call

函数调用三地址码中用于执行调用的指令，通常写明函数名和参数个数。

l-value

左值，表示可存储位置，通常可以出现在赋值号左侧。

r-value

右值，表示表达式的值，通常用于计算或赋值号右侧。

别名

两个或多个名字或表达式可能引用同一存储位置的情况。

SSA

静态单赋值形式。每个变量版本只赋值一次。

phi 函数

SSA 中用于在控制流汇合处合并不同来源变量版本的特殊记号。

phi 展开

把 SSA 中的 phi 函数转换成前驱基本块上的复制或移动指令，使后端低层代码不再需要 phi 记号。

短路求值

布尔表达式中根据左侧结果决定是否计算右侧的求值方式。

布尔表达式的控制流用途

布尔表达式用于 if、while 等语句条件时，其真假通过跳转到不同代码位置体现。

布尔表达式的逻辑值用途

布尔表达式出现在赋值右部等位置时，需要计算出明确的逻辑值，例如 0 或 1。

回填

先生成跳转目标未确定的指令，等目标位置确定后再补上标签或地址。

truelist

布尔表达式为真时应跳转、但目标地址尚未确定的指令位置列表。

falselist

布尔表达式为假时应跳转、但目标地址尚未确定的指令位置列表。

nextlist

语句执行完后应跳到后继语句、但目标地址尚未确定的指令位置列表。

makelist

创建一个只包含某条指令位置的列表，是回填算法使用的辅助函数。

merge

合并两个回填列表的辅助函数。

nextquad

下一条将要生成的三地址指令编号或位置。

M 标记

回填翻译中的空产生式标记，通常用 `M.instr = nextquad` 记录后续代码的起始位置。

N 标记

回填翻译中的空产生式标记，通常生成一个 `goto _` 指令坯，并把该位置放入 `N.nextlist`。

穿越

在语法制导翻译中通过继承属性向下传递未确定跳转目标，等目标位置确定后再配合回填完成跳转。

fall-through

不显式生成跳转，让控制流自然顺序进入下一条指令。

do-while

后测试循环。循环体先执行一次，然后再判断条件。

break

跳出当前循环或 `switch` 结构，控制流转到结构之后。

continue

结束当前循环本轮执行，转到下一轮循环的条件检查位置。